Bazar.com - Distributed Systems Part 1

This project consists of an implementation of a small distributed online book store called Bazar.com based on the micro-services architecture. This was built using Node.js, JavaScript, Express.js, Docker and persistent files called csv.

1. Whole program design:

This project is based on 3 separate micro-services which can all run as separate docker containers.

- **Frontend service** - takes the client request and passes it to the appropriate backend service.
- **Catalog service** - stores all the book's information like their topic, quantity, price etc. This is stored in a .csv file.
- **Order service** - takes a customer's purchase request and stores the orders in a separate .csv file.

2. System Workflow:

All the client's requests would be addressed to the frontend service running on port 3000. It will then pass it to either the catalog or order service depending on the request.

The catalog service:

GET /search/:topic (returns a list of all books on a specific topic)

GET /info/:id (returns information of a specific book)

PUT /update/:id (updates book details)

The order service:

POST /purchase/:id (takes the book id to purchase)

When an order request is placed, the order service would first call the catalog service to check whether the requested book's quantity is greater than 0. Then, it would decrease the quantity by 1 and save it in the orders.csv file.

3. Technologies Used:

Technology Used	Purpose
Node.js	Backend JS runtime environment
Express.js	Backend framework (REST API's)
Docker	Running all services as containerized systems.
CSV Files	Persistent file storage for the books and the orders.
Github	Source Control

4. Design Tradeoffs:

Used simple csv files for the database instead of using a database like MySQL or PostgreSQL. It has been made for simpler implementation and to make it easier to set up. A .csv file is only meant to be used for small applications.

Used Docker to run everything as isolated services and simulate a distributed system. Makes it easily portable, however requires additional setup to implement.

5. Possible Enhancements:

- Replacing the CSV files with a proper database (SQL or NoSQL).
- Implementing a user login and security for purchases.
- Making a proper graphical frontend.
- Adding a caching mechanism to improve performance.
- Making sure simultaneous orders work properly and are synchronized.

6. Known Bugs/ Limitations:

It is impossible for multiple users to make concurrent purchase requests at the same time because a .csv file is not suitable for sophisticated lock mechanisms. There is no authentication in the system and there is no validation to check for purchases that are not allowed.

7. How to Run Program:

1. Open Docker Desktop.
2. Open the project folder in the IDE.
3. Open terminal in the main project folder.
4. Run the following command:

```
docker compose up --build -d
```

5. Open the browser and test:
- http://localhost:3000/search/distributedsystems
- http://localhost:3000/info/2

6. To stop the containers:

```
docker compose down
```